ESR TP2 – Guião do grupo (OTT multimédia)

O que é pedido

O grupo tem de construir, sobre o emulador CORE, uma **rede overlay aplicacional** que entrega vídeo em *streaming* de um ou mais servidores para vários clientes, sem o servidor ter de enviar um fluxo por cliente. Cada nó overlay é uma aplicação (`ott`) que corre num host do CORE; os nós ligam-se entre si por TCP/UDP, calculam rotas por fluxo e reencaminham (e replicam) o vídeo.

Requisitos mínimos do enunciado

- Nó overlay = programa executado à mão num host; pode entrar e sair a qualquer momento.
- Ao arrancar, o nó conhece pelo menos um outro nó (ex.: o servidor como *bootstrapper*) e mantém ligações com X vizinhos (ex.: X=3).
- Cada nó mantém uma tabela de rotas por fluxo: `{Fluxo, Origem, Métrica, Destinos, Estado}`.
- Rotas alternativas e vários clientes em simultâneo; rotas atualizadas quando nós entram/saem ou a rede muda.
- Os nós reenviam os dados segundo a tabela, replicando só onde o caminho se divide (sem tráfego duplicado na mesma ligação).
- **Pelo menos 2 servidores, cada um com pelo menos 2 streams distintas.**
- Usar métricas da rede (saltos, atraso, largura de banda) na construção das rotas.

Avaliação

Item	Peso
Topologia da rede overlay	10%
Gestão dos fluxos de dados	25%
Protocolo aplicacional de controlo	15%
Protocolo de streaming	10%
Solução final integrada (inclui defesa e relatório)	30%
Avaliação intermédia	5% + 5%

Cada item é avaliado em: especificação conceptual, qualidade da implementação, qualidade dos testes e apreciação crítica. A solução final tem ainda defesa/apresentação e relatório.

Entrega (Blackboard, por grupo): `ESR-TP2-PLx-Gy-Rel.pdf` e `ESR-TP2-PLx-Gy-Codigo.zip` (ou `.rar`). O relatório (máx. 12 páginas, formato LNCS) fica para o fim e não está neste guião.

Decisões a fechar na semana 1 (os 3 juntos)

Sugestão de base — o grupo confirma ou altera, mas tem de ficar escrito antes de cada um começar a codificar.

Decisão	Proposta	Porquê
Linguagem	Python 3	O código de apoio do livro (Kurose, `ProgEx.zip`) já está em Python: cliente/servidor RTP e vídeo `movie.Mjpeg`
Transporte de controlo	TCP	Fiabilidade para registo, vizinhos, anúncios e ativação de rotas; a queda da ligação deteta o abandono de um nó
Transporte do vídeo	UDP com n.º de sequência	Tempo real, sem retransmissões; segue o exemplo do livro
Estratégia de overlay	Estratégia 2: `ott <bootstrapper>`	Mais robusta e avaliada como "protocolo de controlo"; o servidor devolve X vizinhos
Estratégia de rotas	Estratégia 1 (servidor anuncia por inundação, cliente ativa)	Cobre exatamente o exemplo do enunciado (rota inativa/ativa, replicação em O2)
Métrica	N.º de saltos na primeira versão; atraso medido (RTT dos vizinhos) como melhoria	Começar simples e evoluir sem mudar o formato da mensagem
Formato das mensagens	Uma linha JSON por mensagem (`{"type":..., ...}`) em TCP, com delimitador `\n`	Fácil de depurar em logs; um módulo partilhado `proto.py`
Repositório	Git, um ramo por elemento, *merge* semanal	Evita conflitos e mostra a contribuição de cada um

Regra de ouro: o formato de cada mensagem (campos e quando é enviada) é definido em conjunto na semana 1 e só muda com acordo dos 3. É o que permite trabalhar em paralelo.

Módulos da solução e interfaces

Cada nó corre o mesmo programa `ott`, com quatro módulos. Cada elemento é dono de um módulo e só fala com os outros pelas interfaces abaixo.

Módulo	Faz	Dono	Expõe aos outros
`proto` + `overlay`	Mensagens, ligações TCP, registo no *bootstrapper*, lista de vizinhos, deteta quedas	Elemento 1	`send(vizinho, msg)`, callback `on_msg(vizinho, msg)`, `vizinhos()`, eventos `vizinho_up/down`
`routing`	Tabela de rotas por fluxo, inundação de anúncios, ativação/desativação, métrica, reencaminhamento perante falhas	Elemento 2	`destinos(fluxo, origem_vizinho)` → lista de vizinhos para onde replicar; `ativar(fluxo)` (cliente)
`stream`	Servidor lê o vídeo e envia pacotes UDP numerados; cliente recebe e mostra; nós intermédios replicam pacotes	Elemento 3	`publicar(fluxo, ficheiro)` (servidor), `receber(fluxo)` (cliente), `forward(pacote)` usa `destinos()`
`testes/` e `core/`	Topologias CORE, scripts de arranque e validação, ferramenta de logs	Elemento 3 (com apoio dos outros)	Cenários 1, 2 e 3 reprodutíveis com um comando

Mensagens de controlo a especificar na semana 1 (campos mínimos entre parênteses):

- `REGISTO` / `VIZINHOS` (nó, lista de vizinhos) — entrar no overlay.
- `HELLO` / `KEEPALIVE` (nó, timestamp) — vivacidade e medição de atraso.
- `ANUNCIO` (servidor, fluxo, métrica, id de sequência) — inundação do servidor.
- `ATIVAR` / `DESATIVAR` (fluxo, cliente) — pedido do cliente, segue o campo Origem.
- `PING` (fluxo, caminho acumulado) — teste de caminhos.
- Dados de vídeo: cabeçalho UDP (fluxo, n.º de sequência, *timestamp*) + *frame* MJPEG.

Divisão de tarefas pelos 3 elementos

A divisão segue o peso da avaliação: o elemento 2 tem o item mais pesado (gestão de fluxos, 25%), por isso os outros dois ficam com mais testes, integração e código de apoio. Os nomes ficam por preencher: **Elemento 1 = _____, Elemento 2 = _____, Elemento 3 = _____**.

Elemento 1 — Overlay e protocolo de controlo (Etapas 0 e 1)

Itens avaliados: topologia overlay (10%) e protocolo aplicacional de controlo (15%).

- **Biblioteca de mensagens** (`proto.py`): codificar/descodificar as mensagens da secção anterior, com validação de campos. Escrever também o documento de especificação das mensagens (servirá ao relatório).
- **Aplicação `ott` base** (Etapa 1): servidor TCP numa porta fixa + ligações de saída, *full-duplex*, com uma *thread* ou `asyncio` por ligação. Testar com troca de `HELLO` entre dois nós.
- **Construção do overlay** com `ott <bootstrapper>` : o nó regista-se; o servidor responde com X vizinhos lidos de um ficheiro de configuração da topologia (`overlay.json` , escrito à mão). Implementar também a Estratégia 1 (`ott <viz1> <viz2>`) como alternativa simples para depuração.
- **Manutenção e monitorização**: `KEEPALIVE` periódico, detecção de vizinho caído (fecho da ligação ou *timeout*), pedido de substituto ao *bootstrapper*, e medição do RTT por vizinho (entrega esse valor ao Elemento 2 como métrica).
- **Apreciação crítica**: testar saída e entrada de nós com o overlay em funcionamento e anotar os resultados.

Elemento 2 — Rotas e gestão de fluxos (Etapas 2 e 3)

Item avaliado: gestão dos fluxos de dados (25%), mais contributo para o protocolo de controlo.

- **Tabela de rotas** (`routing.py`): uma entrada por fluxo com `{Fluxo, Origem, Métrica, Destinos, Estado}` , protegida contra acessos concorrentes (*lock*).
- **Anúncios do servidor**: enviar `ANUNCIO` periodicamente a todos os vizinhos; cada nó soma a métrica da ligação, guarda a melhor origem, ignora anúncios repetidos (id de sequência) e não reenvia para trás. Rota começa **inativa**.
- **Ativação pelo cliente**: `ATIVAR` sobe pelo campo Origem até ao servidor; cada nó acrescenta o vizinho de onde veio a Destinos e passa a Estado=ativa; `DESATIVAR` faz o inverso quando o cliente sai ou o destino fica vazio.
- **Adaptação dinâmica**: se a Origem cai (evento do Elemento 1) ou aparece rota melhor, mudar de origem e reenviar `ATIVAR` pelo novo caminho; expirar rotas sem anúncios.
- **Métricas**: começar com saltos; depois somar o atraso medido. Dois servidores / dois fluxos são apenas mais entradas na tabela (chave = fluxo).
- **Etapa 3 — PING**: o servidor gera `PING` que segue as rotas ativas; cada nó escreve um *log* e acumula o caminho; o cliente imprime o caminho completo. Serve para provar que as rotas funcionam antes do vídeo.

Elemento 3 — Streaming, plano de dados e cenários de teste (Etapa 4 e Etapa 0)

Item avaliado: protocolo de streaming (10%), mais testes e integração da solução final.

- **Servidor de vídeo**: a partir do exemplo do livro, ler `movie.Mjpeg`, empacotar cada *frame* num pacote UDP numerado com cabeçalho (fluxo, sequência, *timestamp*) e enviar ao primeiro nó do overlay. Ter 2 vídeos/streams por servidor (ex.: dois ficheiros ou duas portas).
- **Cliente**: receber pacotes UDP, ordenar por sequência, descartar atrasados e mostrar numa janela (Tkinter, como no livro). Enviar `ATIVAR` ao arrancar, usando a interface do Elemento 2.
- **Plano de dados nos nós**: ao receber um pacote, consultar `destinos(fluxo, origem)` e reenviar uma cópia para cada destino. Garantir que, se houver 2 clientes pelo mesmo ramo, o pacote passa uma só vez nessa ligação.
- **Cenários CORE**: criar as topologias dos cenários 1, 2 e 3 (3 = figura 1 do enunciado), com os *containers* a correr `ott` e um *script* que arranca tudo na ordem certa.
- **Medições**: perdas, atraso e *jitter* no cliente, para a apreciação crítica do streaming.

O que os 3 fazem juntos

- Definir o protocolo e as interfaces na semana 1 (reunião de 1 hora).
- **Revisão cruzada**: o Elemento 1 revê o código do 2, o 2 revê o do 3 e o 3 revê o do 1 (um *pull request* por semana).
- Cada um **escreve e corre os testes do seu módulo** (subitem c da avaliação) e **prepara a parte da defesa** que lhe corresponde: todos têm de saber explicar todo o sistema.
- Integração e ensaio final dos cenários, em conjunto.

Calendário (9 semanas)

Os dois momentos de avaliação intermédia (5% + 5%) não têm data no enunciado: confirmar com o docente. O plano assume a primeira por volta da semana 4 e a segunda por volta da semana 6, por isso cada uma deve ter algo funcional para mostrar.

Semana	Elemento 1 — Overlay	Elemento 2 — Rotas	Elemento 3 — Streaming e testes
1	Reunião de grupo: decisões e formato das mensagens. `proto.py` e `ott` com `HELLO` entre 2 nós	Estudar o enunciado e desenhar a tabela de rotas e o fluxo de mensagens (esboço em papel)	Montar CORE, criar cenário 1 e rodar os exemplos do livro (`ProgEx.zip`)

Semana	Elemento 1 — Overlay	Elemento 2 — Rotas	Elemento 3 — Streaming e testes
2	Ligações full-duplex, vários vizinhos, Estratégia 1 (`ott viz1 viz2`)	Tabela de rotas + `ANUNCIO` com inundação controlada, em testes locais sem rede	Servidor de vídeo UDP e cliente a mostrar `movie.Mjpeg` direto, sem overlay
3	`ott <bootstrapper>` e `overlay.json` com X=2 ou 3 vizinhos	`ATIVAR` / `DESATIVAR` e estado inativo/ativo; integrar com o overlay do E1	Cenário 2 no CORE e *script* de arranque; cabeçalho com fluxo e sequência
4	**Intermédia 1:** overlay no cenário 1/2 com `KEEPALIVE` e deteta quedas	**Intermédia 1:** anúncio + ativação em 2 clientes; `PING` a mostrar o caminho	**Intermédia 1:** plano de dados: reencaminhar e replicar o vídeo pelo overlay no cenário 1
5	Saída e entrada de nós com o overlay ativo; pedido de substituto de vizinho	Rotas alternativas; mudar de origem quando um vizinho cai	Cenário 2: 2 clientes, replicação no nó certo; medir perdas e atraso
6	RTT por vizinho e entrega ao E2	Métrica por atraso (RTT) em vez de saltos	2 servidores × 2 streams; cenário 3 (figura 1)
7	Testes de robustez e limpeza do código; documentar o protocolo de controlo	Testes de falhas de rotas e carga; documentar o algoritmo	Testes de streaming (perdas, *jitter*) e documentar o protocolo de dados
8	Integração completa dos 3 módulos; correção de erros nos 3 cenários (em conjunto)	Idem	Idem; preparar demonstração e guiar a “execução de referência”
9	Ensaio da defesa, congelar o código, gerar o `.zip`	Idem	Idem; o relatório começa aqui (fora deste guião)

Testes e checklist de entrega

Os três cenários do enunciado são a base dos testes. Para cada um, guardar *logs* e capturas, que servem depois ao relatório.

Cenário	Topologia	O que provar
1	1 servidor, 2 clientes, 1 nó intermédio	Replicação no nó intermédio: o servidor envia 1 fluxo e os 2 clientes recebem
2	1 servidor, 2 clientes, 3 intermédios com caminhos alternativos	Escolha da melhor rota, mudança de rota quando um nó cai
3	Figura 1: 11 nós, 5 clientes (C1–C5), S1 = O1	Vários clientes em simultâneo, entrada/saída de clientes, 2 servidores × 2 streams

Testes que não podem faltar

- Entrada e saída de um nó intermédio durante o streaming.
- Cliente liga e desliga (rota passa a ativa e depois inativa).
- Falha de uma ligação (no CORE: desativar o link) e recuperação por rota alternativa.
- Dois servidores e dois streams ao mesmo tempo, sem tráfego duplicado na mesma ligação.
- Perdas e atraso medidos no cliente com ligações com perdas (parâmetros do CORE).

Checklist de entrega (sem relatório)

- [ ] Código organizado por módulos, com `README` de como arrancar cada cenário.
- [ ] Ficheiros de topologia CORE (`.imn`) e *scripts* incluídos no `.zip` .
- [ ] Cenários 1, 2 e 3 correm do zero com um comando.
- [ ] Cada elemento sabe explicar todo o sistema (a defesa pergunta a qualquer um).
- [ ] `ESR-TP2-PLx-Gy-Codigo.zip` gerado e carregado no Blackboard (na “troca de ficheiros” do grupo).
- [ ] O relatório fica para o fim, com os resultados e capturas guardados ao longo das semanas.

Dúvidas para o docente

- Datas e conteúdos esperados das duas avaliações intermédias (5% + 5%).
- Podemos usar Python em vez de Java, e TCP para controlo com UDP para vídeo?
- Quantos vizinhos X por nó e quantos nós overlay esperam no cenário 3?
- Basta o servidor conhecer o overlay por ficheiro de configuração escrito à mão?
- Dois servidores: são dois nós overlay distintos com anúncios independentes?